
Prompt Engineering & In-Context Learning

Programming with English: From "Magic Words" to Robust Systems

The New Paradigm

The Mental Shift (2023 vs. 2025)

- **2023 Mindset (The "Whisperer"):** "If I ask politely and use the word 'please', the AI works better." (Subjective, fragile).
- **2025 Mindset (The Engineer):** "I need to optimize the context window, structure the output schema (JSON/XML), and evaluate reliability." (Objective, robust).
- **Key Concept: In-Context Learning.** The model isn't "learning" permanently (like fine-tuning); it is "learning" **temporarily** from the data you put in the prompt.

Prompt Engineering is "Soft Programming"

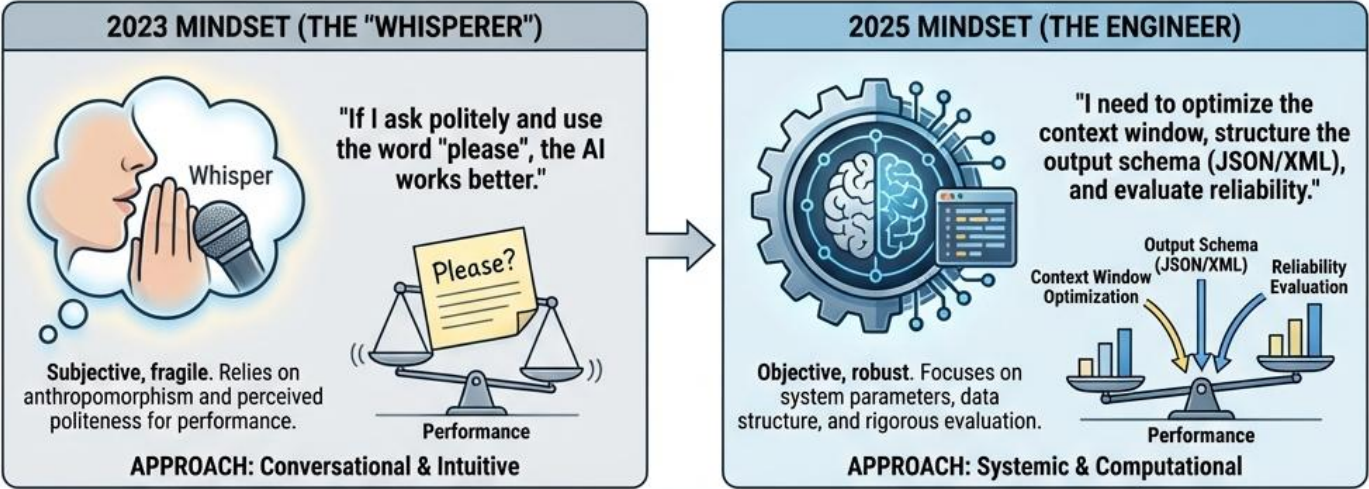
- **Analogy:**
 - **Variables** = The Context (Documents, User History).
 - **Functions** = The Instructions ("Summarize this", "Extract entities").
 - **Return Type** = The Output Format (JSON, List, Markdown).
- **The Code:** In standard programming, you write `function(x)`. In Prompt Engineering, you write:
- "You are a function that takes 'x' and returns 'y'. Do not output anything else."

The 4 Components of a Perfect Prompt

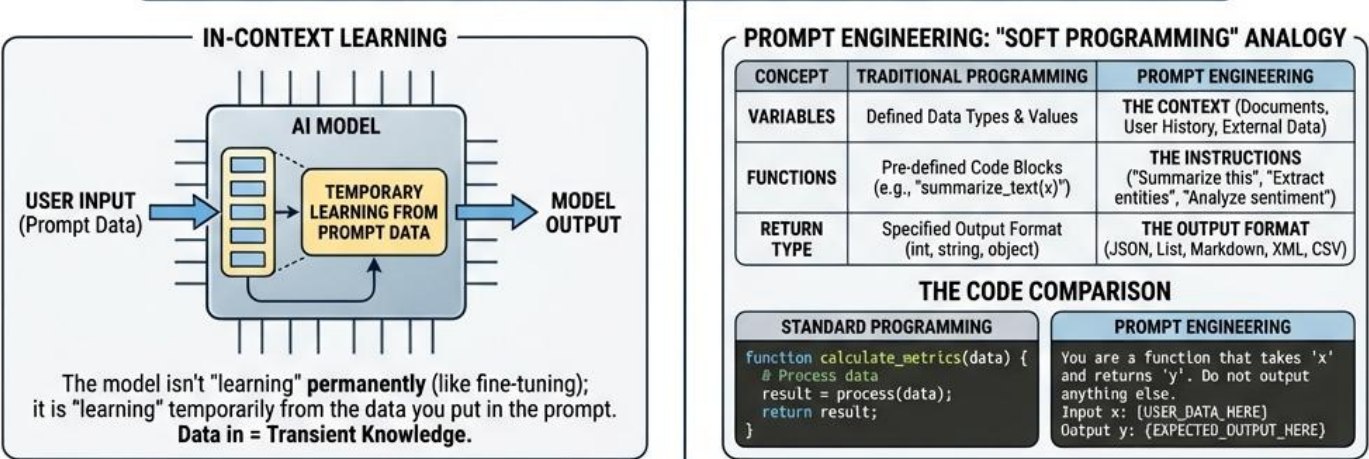
(there are many actually)

- **Role/Persona:** "You are a Senior Robotics Engineer..." (Sets the latent space).
- **Instruction:** "Analyze the following log file..." (The core task).
- **Context/Constraint:** "Use only the data provided below. Output in valid JSON." (Guardrails)
- **Few-Shot Examples:** "Here is an example of a good input and output..." (The gold standard).

THE MENTAL SHIFT (2023 VS. 2025): PROMPT ENGINEERING & IN-CONTEXT LEARNING

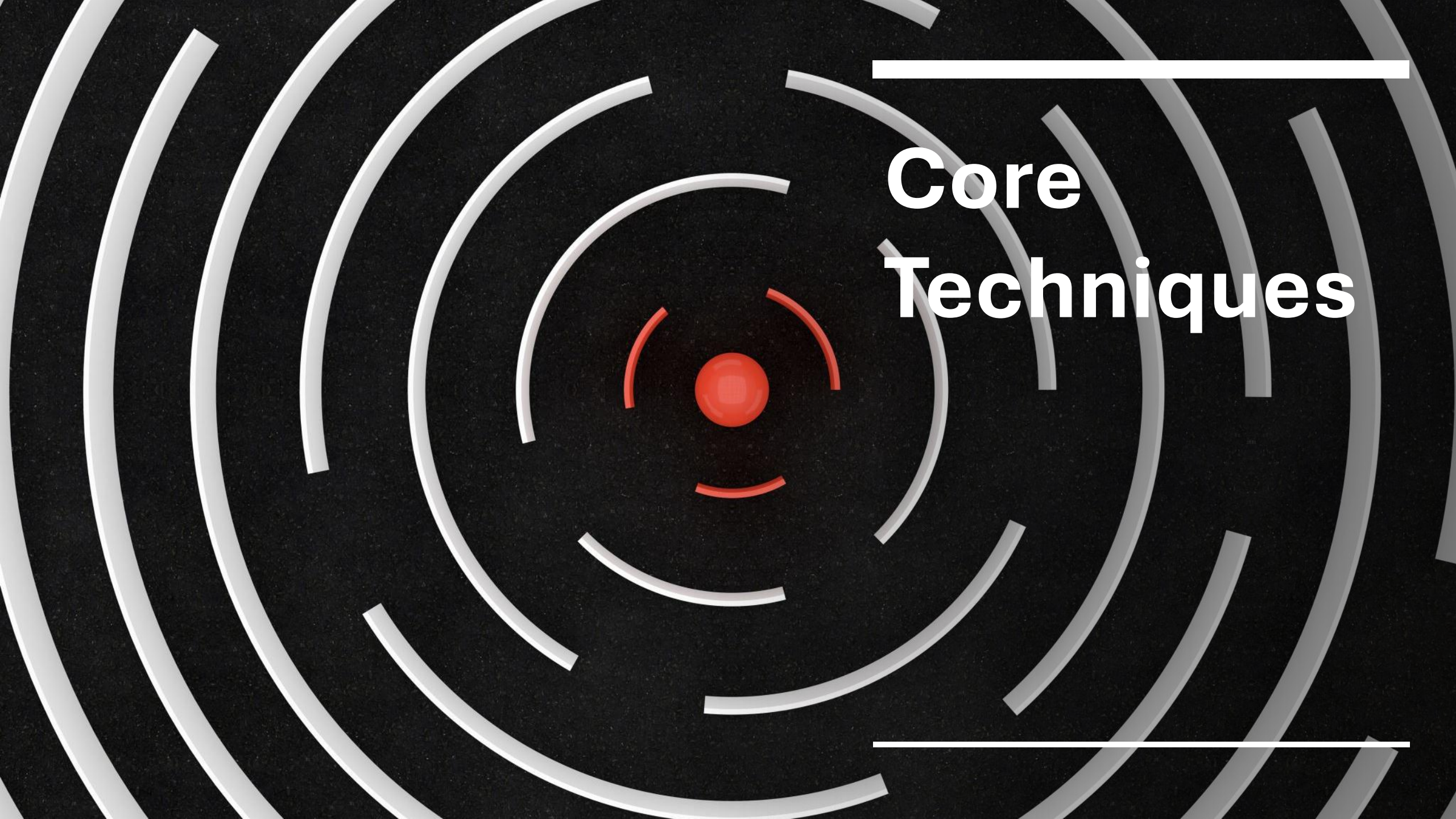


KEY CONCEPT: IN-CONTEXT LEARNING & PROMPT ENGINEERING AS "SOFT PROGRAMMING"



THE 4 COMPONENTS OF A PERFECT PROMPT



The background features a series of concentric, broken circles in shades of gray, creating a ripple effect. At the center is a solid red sphere, surrounded by three small red arcs. Two horizontal white lines are positioned above and below the text.

Core Techniques

Level 1: Zero-Shot vs. Few-Shot

- **Zero-Shot:** Asking blindly.
 - *Input:* "Classify this review: 'The battery died in 5 minutes.'" -> *Output:* "Negative."
- **Few-Shot (In-Context Learning):** Giving examples to set the pattern.
 - *Input:*
 - "Review: 'Great motor.' -> Label: Hardware_Positive"
 - "Review: 'Software crashed.' -> Label: Software_Negative"
 - "Review: 'The battery died.' -> Label: ?"
 - *Why it wins:* It teaches the *format* and the *nuance* instantly without training.

Level 2: Chain-of-Thought (CoT)

- **The Breakthrough:** LLMs are bad at math/logic if they answer immediately. They need "thinking time."
- **The Technique:** Force the model to generate intermediate steps.
- **Standard Prompt:** "How many wheels do 3 cars and 2 bikes have?" -> *Model might guess.*
- **CoT Prompt:** "Think step-by-step. First count car wheels, then bike wheels, then sum them."
 - Step 1: 3 cars * 4 = 12.
 - Step 2: 2 bikes * 2 = 4.
 - Step 3: 12 + 4 = 16.

Level 3: Tree-of-Thoughts & Self-Consistency

- **Advanced Reasoning (2025 Standard):**
 - **Self-Consistency:** Ask the same question 5 times. If 4 answers are "A" and 1 is "B", pick "A".
 - **Tree-of-Thoughts:** Ask the model to *generate three possible plans, critique each one, and pick the best path before solving.*
- **Robotics Use Case:** "Plan a path for the robot. Path A is short but rocky. Path B is long but smooth. Evaluate risks of both."

Level 4: Structured Output (JSON Mode)

- **The Problem:** You can't code with "Sure! Here is the list..."
- **The Fix:** [Enforcing Schemas](#).
- **Prompt:** "Output strictly in JSON format matching this schema: { 'sentiment': float, 'entities': list }."
- **Why:** This allows you to plug the LLM directly into Python code (`json.loads(response)`).

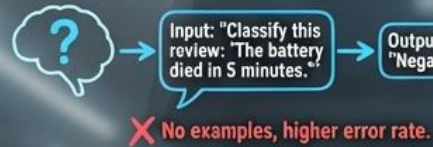
"Context Engineering" (The Window)

- **The "Needle in a Haystack" Problem:** Putting important info in the middle of a long prompt often gets lost.
- **Best Practice:**
 - System Instructions (Start).
 - **Context/Data** (Middle).
 - Specific Task/Question (End - "Recency Bias" helps here).

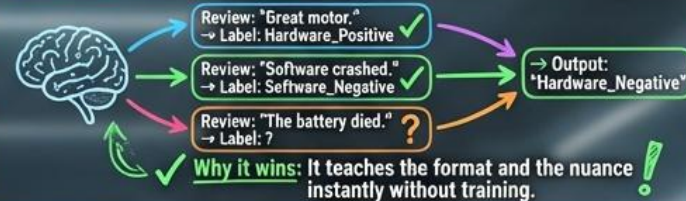
MODERN PROMPT ENGINEERING:

From Zero-Shot to Structured Thought

Zero-Shot: Asking blindly

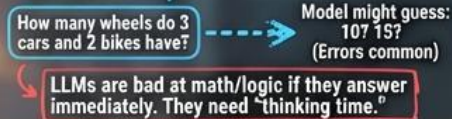


Few-Shot (In-Context Learning): Giving examples to set the pattern



Chain-of-Thought (CoT): The Breakthrough

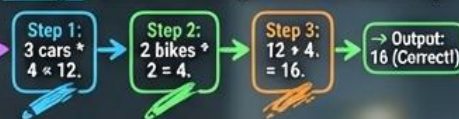
Standard Prompt



CoT Prompt

Think step-by-step. First count car wheels, then bike wheels, then sum them.

Technique: Force the model to generate intermediate steps.



Self-Consistency: Ask the same question 5 times



Tree-of-Thoughts: Ask the model to generate three possible plans, critique each one, and pick the best path before solving.

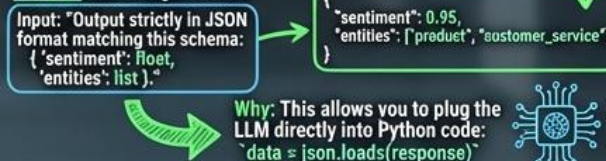


Structured Output (JSON Mode): The Problem vs. The Fix

The Problem: You can't code with "Sure! Here is the list..."



The Fix: Enforcing Schemas



Context Engineering (The Window): The "Needle in a Haystack" Problem & Best Practice

Context Window:



Best Practice:

System Instructions (Blue: Set Behavior)

Context/Data (Orange: Provide Background)

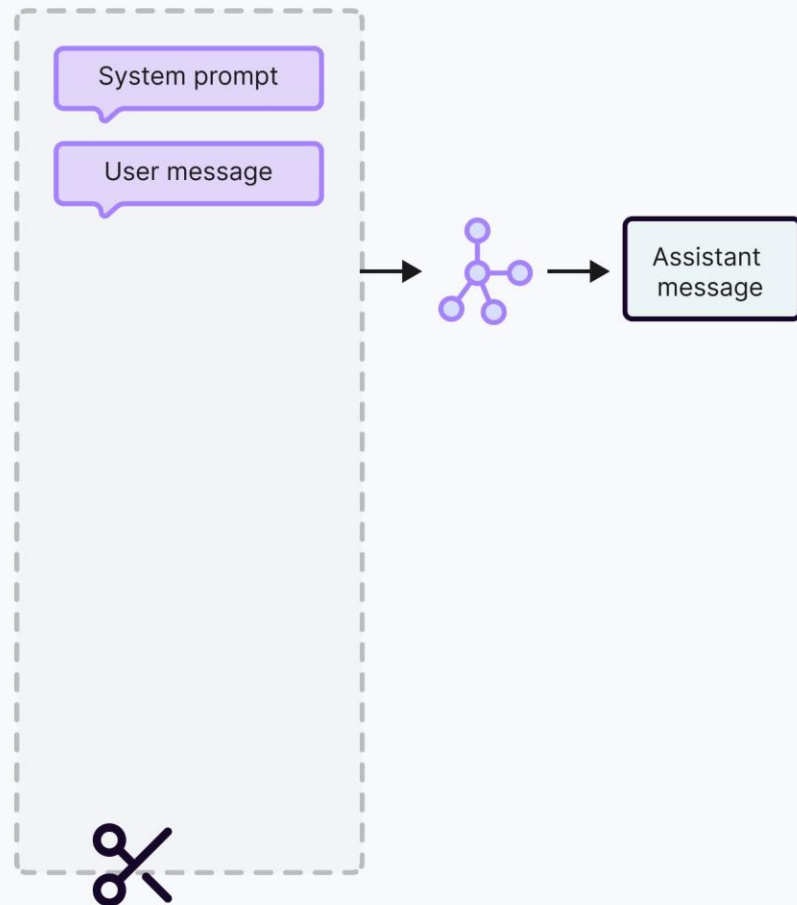
Specific Task/Question (Green: Recency Bias Priority!)

Optimized Output

Prompt vs. Context Engineering

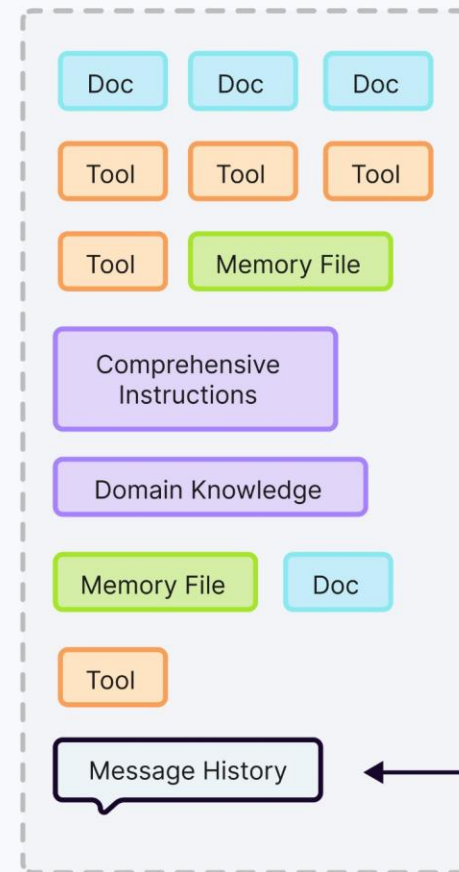
Simple Prompt Engineering

Context window

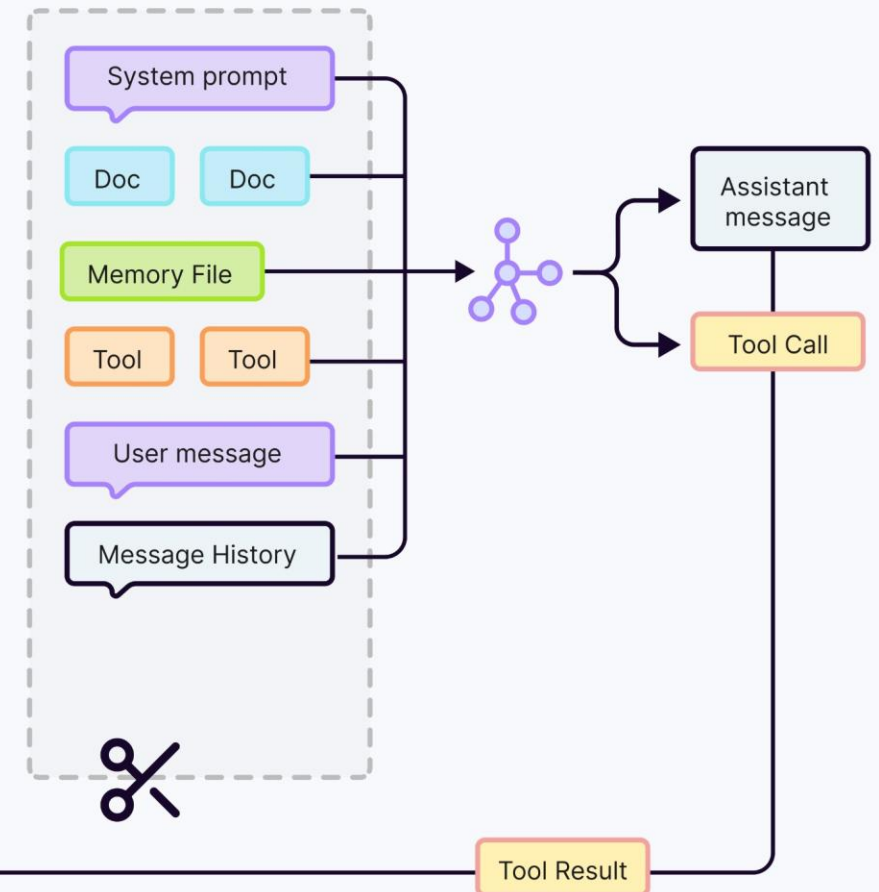


Context Engineering

Possible context to give model



Context window



The Frontier – DSPy & Programming

"Don't Write Prompts, Compile Them"

- **Introduction to [DSPy](#) (Declarative Self-Improving Python):** A Stanford framework that is replacing manual prompt engineering.
- **The Concept:**
 - You define the **Goal**: "Answer questions based on context."
 - You define the **Metric**: "Answer must match the fact sheet."
 - **DSPy Optimizes the Prompt:** It acts like a compiler. It tries 50 different variations of the prompt to see which one gets the highest score, then saves the best one.

System 2 Reasoning Models

- **New in 2025:** Models that *automatically* do Chain-of-Thought without you asking. OpenAI – “o” models, GPT5, GPT-5.1, Google Gemini3, Claude-Sonnet-4-5, Claude-Opus-4-5, DeepSeek, Kimi-Max and many more.
- **Implication:** For these models, "simple" prompts are often better. The model manages its own thinking process.



Reasoning Techniques

Pre-Training

Fine-tuning

Mixture of Experts (MoE)

Alignment Training

In-Context Learning

Autonomous Agent

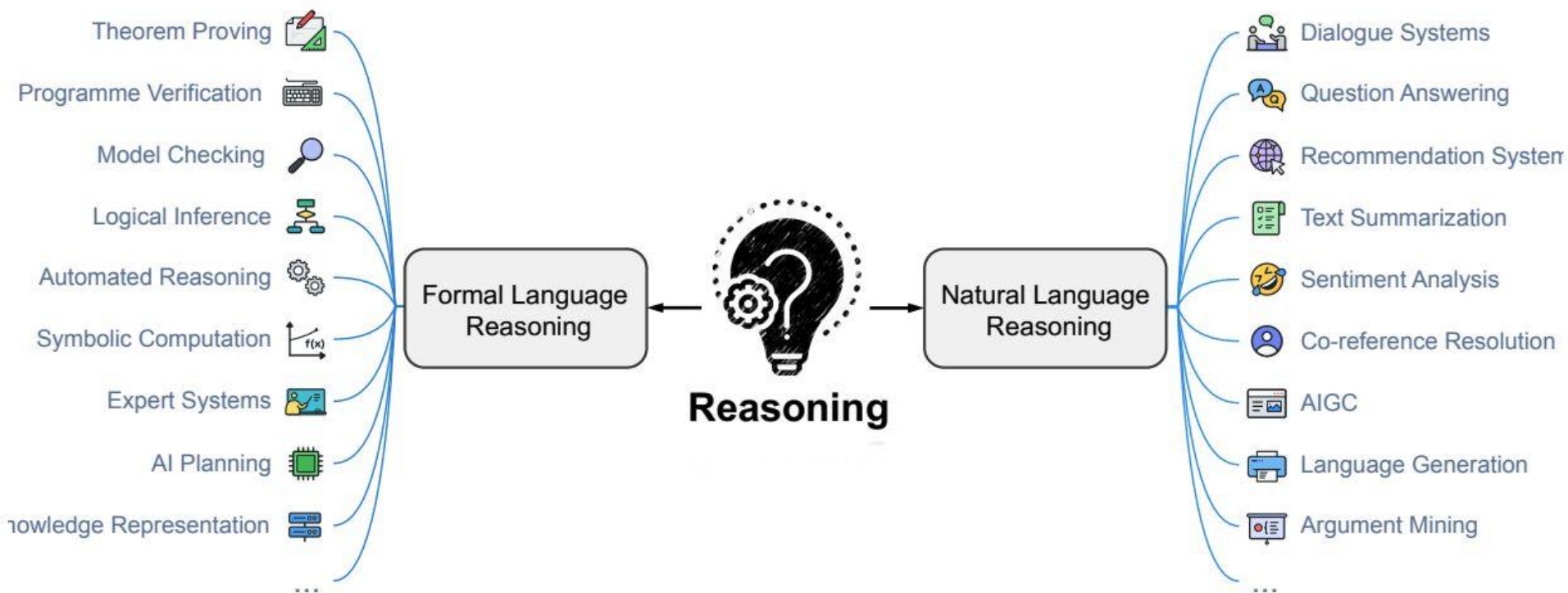


Fig. 1: Two broad types of language reasoning and examples of the supported tasks.

Example	
Fact1	This animal is a robin.
Rule	All robins are birds.
Fact2	This animal is a bird.
Reasoning Type	Representation
Deduction	(Fact1 + Rule $\rightarrow$ Fact2)
Abduction	(Fact1 + Rule $\leftarrow$ Fact2)
Induction	(Fact1 + Fact2 $\rightarrow$ Rule)

Table 3: Illustration of deductive reasoning, abductive reasoning, and inductive reasoning. In this example, the black text represents the given knowledge, while the red text represents the inferred knowledge. The term “Fact” indicates specific information, while “Rule” denotes a general principle or guideline.

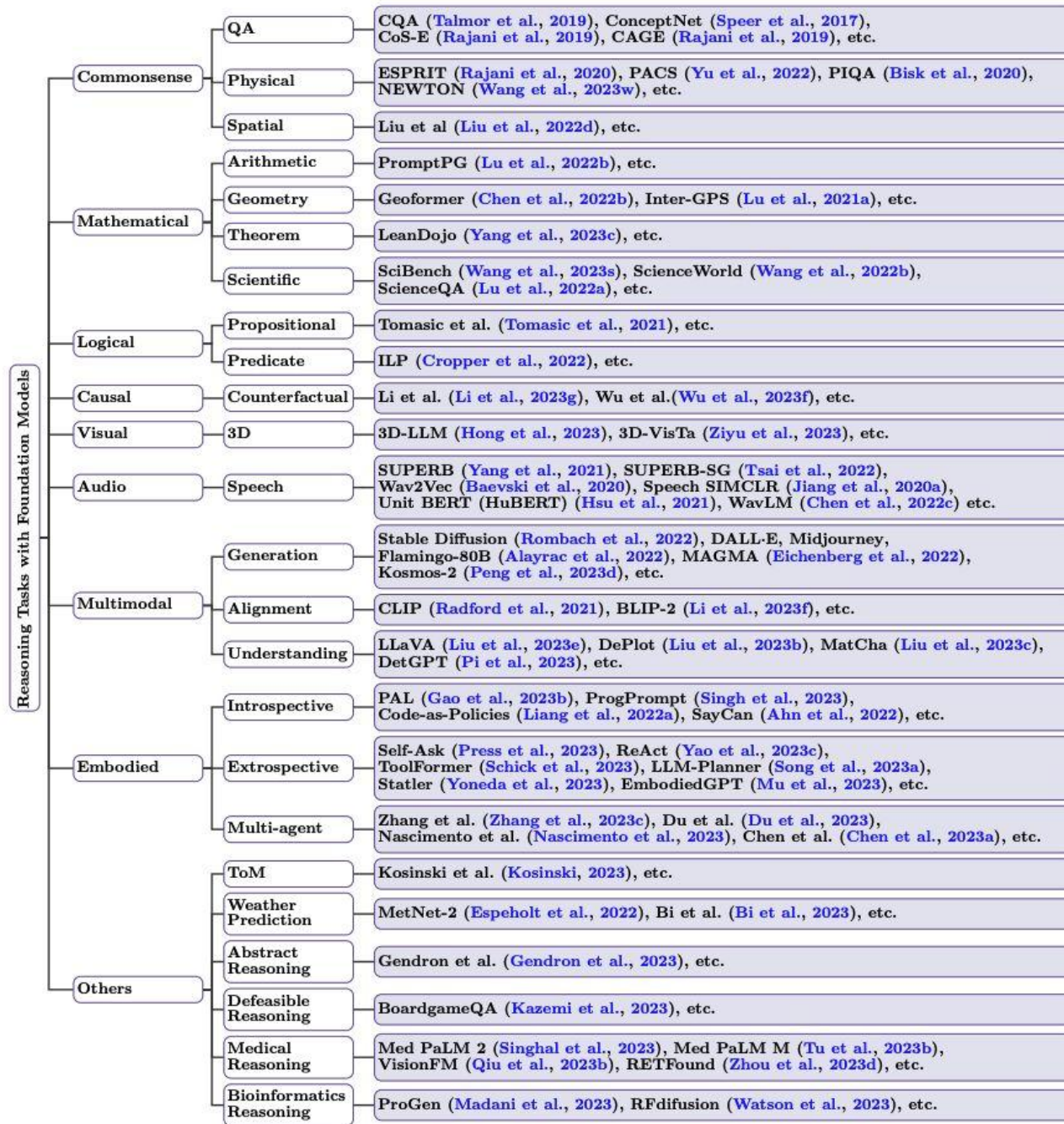


Fig. 5: Taxonomy of Reasoning Tasks with Foundation Models. Only the representative approaches for each type of task are listed.

Lab



Prompting & LLM as a Judge

Prompt Battle (LMArena or any other provider)

- **The Mission:** You have a dataset of 50 "tricky" customer service emails (sarcasm, mixed languages, subtle complaints) or any other dataset. Note: HuggingFace datasets.
- **The Goal:** Extract the **Core Issue** and **Sentiment** with 100% accuracy.
- **The Constraint:** You cannot change the model (e.g., Llama 3.2 or GPT-5-mini). You can only change the *Prompt*. [Use your own LLM model]

LLM as a Judge

- **How we grade:** We won't grade manually. We will use LLM as the "Judge."
- **The Rubric (Code Provided):**
 - *Correctness (1-5):* Did it identify the right product?
 - *Format (Pass/Fail):* Is it valid JSON?
 - *Reasoning:* Did it explain why?
- **Why this matters:** This is how industry evaluates AI systems at scale